FOUNDRY

Foundry Introduction

Module - 1

Foundry

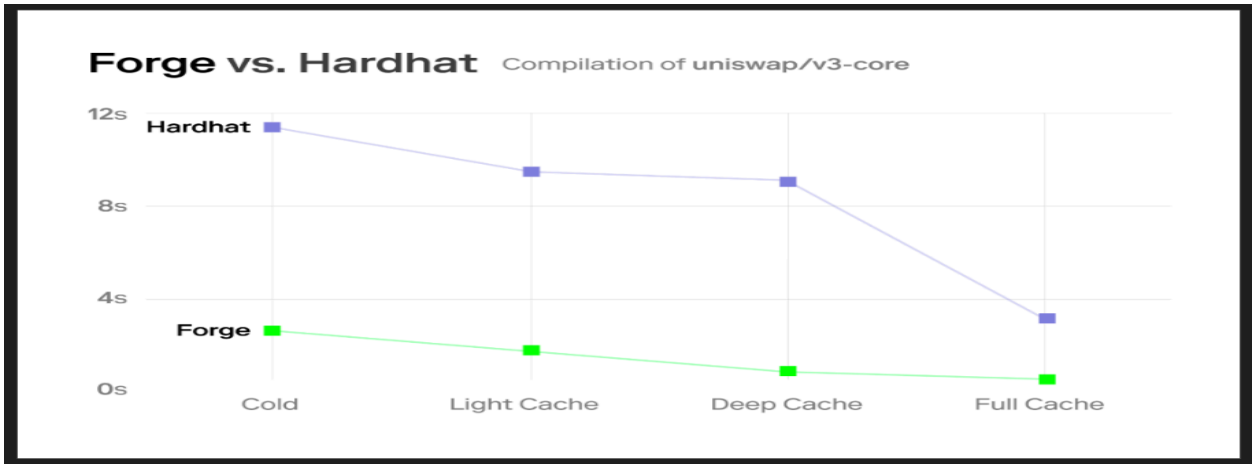
Foundry typically refers to a development framework or toolkit for building and testing smart contracts, particularly in the Ethereum ecosystem. Foundry simplifies the process of writing, deploying, and testing smart contracts by offering a suite of tools and libraries designed to streamline development workflows.

Foundry consists of:

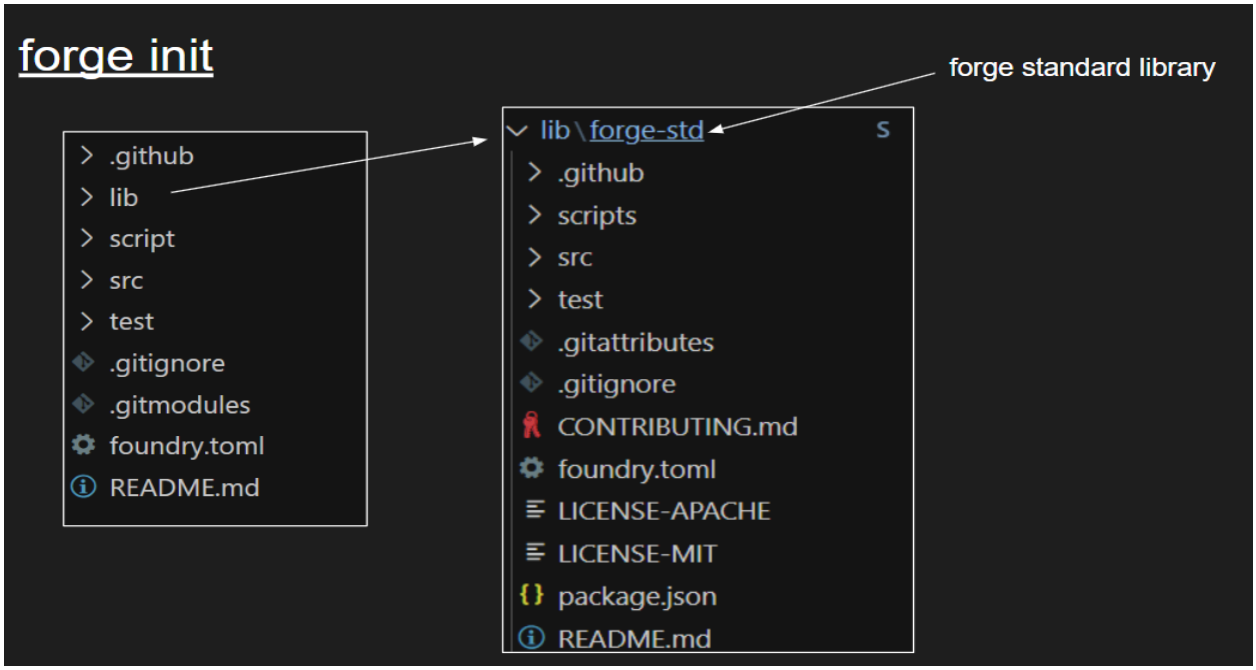
- Forge: Ethereum testing framework (like Truffle, Hardhat and DappTools).
- Cast: Swiss army knife for interacting with EVM smart contracts, sending transactions and getting chain data.
- Anvil: Local Ethereum node, akin to Ganache, Hardhat Network.
- Chisel: Fast, utilitarian, and verbose solidity REPL.

Testing Benchmarks

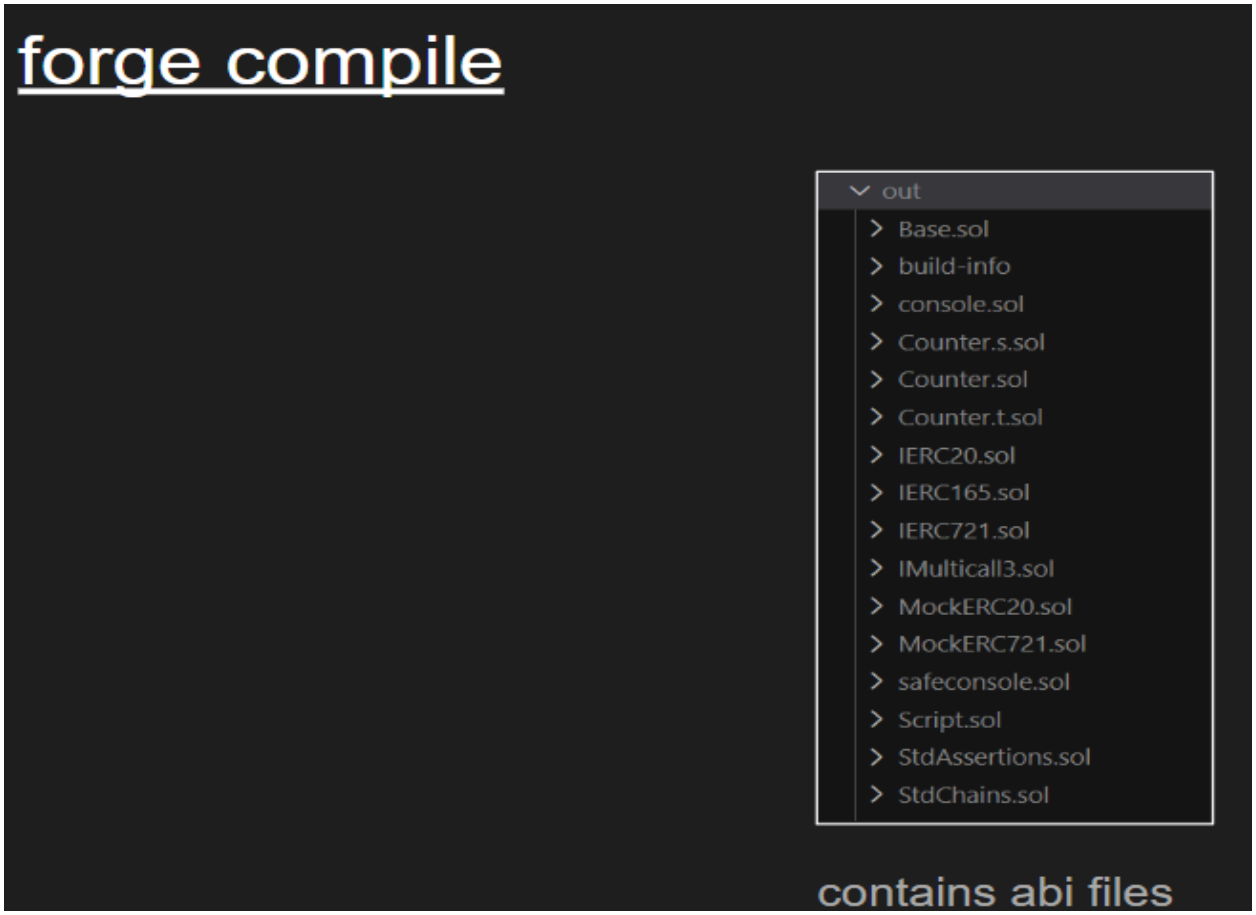
Project	Forge	DappTools	Speedup
transmissions11/solmate	2.8s	6m34s	140x
reflexer-labs/geb	0.4s	23s	57.5x
Rari-Capital/vaults	0.28s	6.5s	23x



Feature	Foundry	Hardhat
Performance and Speed	Fast, Rust-based, highly efficient for large projects and testing.	Slower, JavaScript-based, may lag with complex deployments.
Testing Framework	Built-in **Forge** with Solidity-native syntax, no additional setup required.	Uses Mocha/Chai (JavaScript), flexible but may feel less integrated for Solidity-focused testing.
Gas Optimization	Automatic, integrated gas snapshots and reports.	Requires setup with plugins like `hardhat-gas-reporter`.
Debugging Tools	Built-in debugger with step-through execution, faster and more integrated.	Includes debugging tool, but less efficient compared to Foundry.
Focus on Solidity	Primarily Solidity-focused, optimized for Ethereum development.	Generalized with JavaScript support, suitable for multi-chain projects.
Modularity and Tool Integration	Integrates well with Ethereum-specific tools like **Ethers.js** and **Slither** for analysis.	Flexible but more plugin-dependent, can have compatibility issues.
Developer Experience (CLI)	Intuitive, terminal-friendly CLI with minimal setup and configuration.	Verbose CLI with more configuration needed (`hardhat.config.js`).
Storage Layout & Symbolic Execution	Native support for **symbolic execution** and detailed **storage layout** insights.	Lacks native support for symbolic execution, requires third-party tools.
Community and Updates	Rapidly evolving, strong Ethereum-focused community with frequent updates from Paradigm.	Established, large community with extensive resources for new users.
JavaScript Ecosystem	Less JavaScript-centric; better for pure Solidity developers.	Integrated with JavaScript, ideal for those needing front-end compatibility.
Plugin Ecosystem	Lightweight but still growing; focuses on core Ethereum features.	Extensive plugin ecosystem, supports multi-chain projects and flexible workflows.



>>>forge init



>>>forge compile

TOML file configuration

```
[profile.default]
src = "src"             # Sets the directory for source files. Foundry will look for contracts in the "src" folder.
out = "out"             # Sets the directory where compiled output (e.g., ABIs and bytecode) will be stored.
libs = ["lib"]          # Specifies additional directories for libraries or dependencies.
solc-version = "0.8.20" # Specify the desired Solidity version
```

TOML configuration

The following is a foundry.toml file with all configuration options set. See also /config/src/lib.rs and /cli/tests/it/config.rs.

```
## defaults for _all_ profiles
[profile.default]
src = 'src'
test = 'test'
script = 'script'
out = 'out'
libs = ['lib']
auto_detect_remappings = true # recursive auto-detection of remappings
remappings = []
# list of libraries to link in the form of `<path to lib>:<lib name>:<address>`: `"src/MyLib.sol:MyLib:0x8
# the <path to lib> supports remappings
libraries = []
cache = true
cache_path = 'cache'
broadcast = 'broadcast'
# additional solc allow paths
allow_paths = []
```

>>>https://github.com/foundry-rs/foundry/tree/master/crates/config(for TOML configurations)

anvil

```
ksh@DESKTOP-448NKKT:/mnt/c/Projects/foundry/foundry-demo$ anvil

                                _   _
                               (_) | |
      __  _   _ __   __   __  _  | |
     / _` | | '_ \  \ \ / / | | | |
    | (_| | | | | |  \ V /  | | | |
     \__,_| |_| |_|   \_/   |_| |_|

    0.2.0 (f3376a6 2024-11-09T00:21:21.149860638Z)
    https://github.com/foundry-rs/foundry

Available Accounts
==================

(0) 0xf39Fd6e51aad88F6F4ce6aB8827279cfffb92266 (10000.000000000000000000 ETH)
(1) 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (10000.000000000000000000 ETH)
(2) 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC (10000.000000000000000000 ETH)
(3) 0x90F79bf6EB2c4f870365E785982E1f101E93b906 (10000.000000000000000000 ETH)
(4) 0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65 (10000.000000000000000000 ETH)
(5) 0x9965507D1a55bcC2695C58ba16FB37d819B0A4dc (10000.000000000000000000 ETH)
(6) 0x976EA74026E726554dB657fA54763abd0C3a0aa9 (10000.000000000000000000 ETH)
(7) 0x14dC79964da2C08b23698B3D3cc7Ca32193d9955 (10000.000000000000000000 ETH)
(8) 0x23618e81E3f5cdF7f54C3d65f7FBc0aBf5B21E8f (10000.000000000000000000 ETH)
(9) 0xa0Ee7A142d267C1f36714E4a8F75612F20a79720 (10000.000000000000000000 ETH)

Private Keys
==================

(0) 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
```

To run a local development

Deploying A Contract

```
forge create Counter --rpc-url http://127.0.0.1:8545 --interactive
```

To clear the terminal history

```
history - c
```

```
ksh@DESKTOP-448NKKT:/mnt/c/Projects/foundry/foundry-demo$ forge
create Counter --rpc-url http://127.0.0.1:8545 --interactive
[⠊] Compiling...
No files changed, compilation skipped
Enter private key:
Deployer: 0xf39Fd6e51aad88F6F4ce6aB8827279cfffFb92266
Deployed to: 0x5FbDB2315678afecb367f032d93F642f64180aa3
Transaction hash: 0x755c8a33e320ce0cdc2360e4cc2b86cb70663b0f5187
6abdb1f5ab14c216dd77
```

Note - Use your own terminal for private key pasting because terminals of VS code might send the data to their server for training purposes.

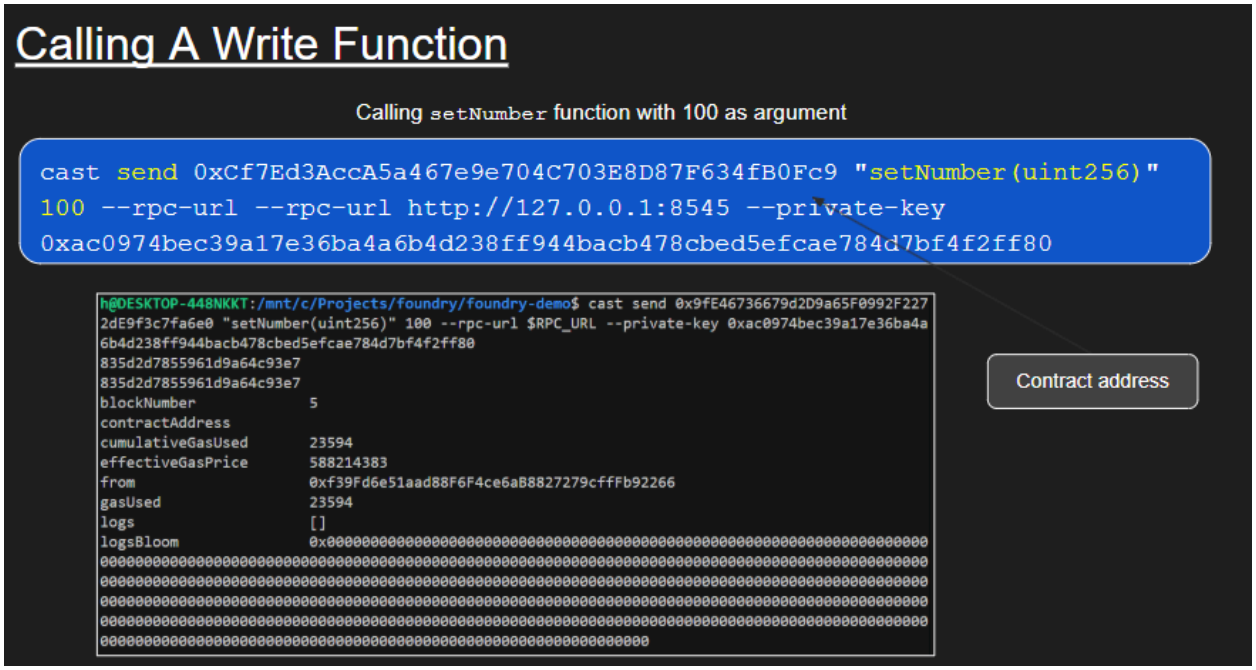
>>>forge create Counter --rpc-url http://127.0.0.1:8545 --interactive
>>>forge create Counter --rpc-url http://127.0.0.1:8545 --interactive --broadcast

Writing a script in solidity for deployment of contract

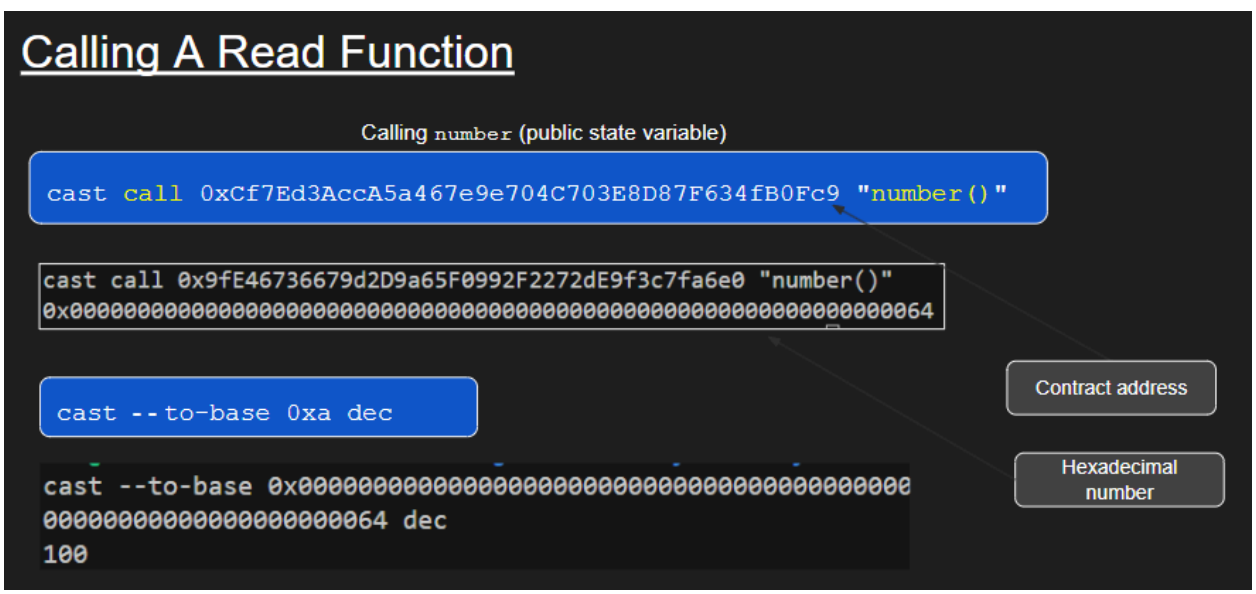
"s" is used as convention to write the script file. Counter.s.sol

```
forge script script/Counter.s.sol --rpc-url http://127.0.0.1:8545
--broadcast --private-key
0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
```

>>>forge script script/Counter.s.sol --rpc-url http://127.0.0.1:8545 --broadcast --private-key 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80

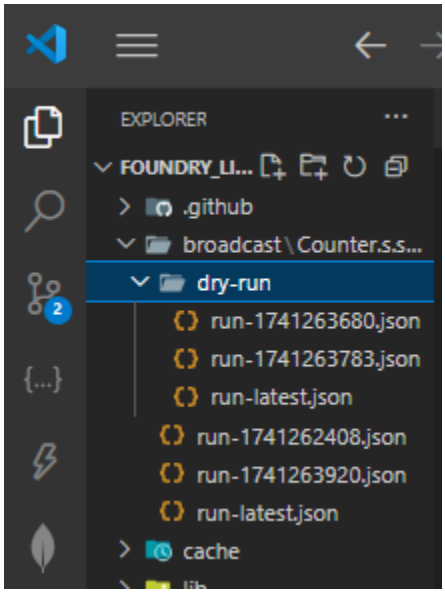


>>>cast send 0x5FbDB2315678afecb367f032d93F642f64180aa3 "setNumber(uint256)" 100 --rpc-url http://127.0.0.1:8545 --private-key 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80



>>>cast call 0x5FbDB2315678afecb367f032d93F642f64180aa3 "number()"
>>>cast to-dec <hex_number>

>>>forge script script/Counter.s.sol --rpc-url http://127.0.0.1:8545 --private-key 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
// - -broadcast removed on above command



Counter.s.sol

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.13;

import {Script, console} from "forge-std/Script.sol";
import {Counter} from "../src/Counter.sol";

contract CounterScript is Script {
    Counter public counter;

    function setUp() public {}

    function run() public {
        vm.startBroadcast();

        counter = new Counter();

        vm.stopBroadcast();
    }
}
```

The `setUp` function is an empty function in this example. In testing or deployment scripts, `setUp` is often used to initialize any preconditions or setup logic, but it's empty here.

The `run` function is the main function of the script. It is the function that gets executed when you run the script (typically via `forge script`).

`vm.startBroadcast()` starts a "broadcast" to deploy transactions to the blockchain. This is part of Foundry's tooling, which allows you to simulate or send actual transactions to the network.

`counter = new Counter();` deploys the `Counter` contract to the blockchain and assigns the deployed contract instance to the `counter` variable. The `Counter` contract is assumed to have a constructor that takes no arguments

`vm.stopBroadcast()` ends the broadcasting session, committing the deployment transaction to the blockchain.

Simulation mode

```
## Setting up 1 EVM.

==========================

Chain 31337

Chain 31337


Estimated gas price: 2.000000001 gwei

Estimated gas price: 2.000000001 gwei

Estimated total gas used for script: 138719

Estimated amount required: 0.000277438000138719 ETH
Estimated total gas used for script: 138719

Estimated amount required: 0.000277438000138719 ETH

Estimated amount required: 0.000277438000138719 ETH

==========================

SIMULATION COMPLETE. To broadcast these transactions, add --broadcast and wallet configur
ation(s) to the previous command. See forge script --help for more.

Transactions saved to: /mnt/c/Projects/foundryDemo/broadcast/Counter.s.sol/31337/dry-run/
run-latest.json

Sensitive values saved to: /mnt/c/Projects/foundryDemo/cache/Counter.s.sol/31337/dry-run/
run-latest.json
```

Simulation Mode: The transactions were not sent to the actual blockchain. Instead, Foundry simulated the execution on a local virtual environment, estimating the gas usage, required funds, and other details.

```
forge script script/Counter.s.sol --rpc-url http://127.0.0.1:8545 --broadcast
--private-key 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
```

.env file (not advisable)

```
.env
PRIVATE_KEY=0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
RPC_URL=http://127.0.0.1:8545
```

```
ksh@DESKTOP-448NKKT:/mnt/c/Projects/foundry/foundry-demo$ source .env
ksh@DESKTOP-448NKKT:/mnt/c/Projects/foundry/foundry-demo$ echo $PRIVATE_KEY
0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
ksh@DESKTOP-448NKKT:/mnt/c/Projects/foundry/foundry-demo$ echo $RPC_URL
http://127.0.0.1:8545
```

```
ksh@DESKTOP-448NKKT:/mnt/c/Projects/foundry/foundry-demo$ forge script script/Counter.s.
sol --rpc-url $RPC_URL --broadcast --private-key $PRIVATE_KEY
```

Use export in Shell (not advisable)

```
export RPC_URL=http://127.0.0.1:8545
export PRIVATE_KEY=0xabc123...your_private_key_here
```

Private Key via Foundry

The command cast wallet import defaultKey --interactive is used to securely import a private key into the cast wallet by naming it defaultKey. The --interactive flag prompts the user to input the private key manually, ensuring it isn't exposed via command-line arguments. This key can then be referenced by name (defaultKey) in subsequent forge or cast commands, streamlining tasks like sending transactions or signing messages.

```
cast wallet import defaultKey --interactive
```

Deploying contract:

```
forge script script/Counter.s.sol --rpc-url http://127.0.0.1:8545
--account defaultKey --sender
0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266 --broadcast
```

>>>cast wallet import defaultKey --interactive

>>>forge script script/Counter.s.sol --rpc-url http://127.0.0.1:8545 --account defaultKey --sender 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266 --broadcast

Run this command to find the correct directory:

>>echo $HOME

Then, navigate to Foundry's keystore location:

>>ls ~/.foundry/keystores/

If `defaultKey` exists, remove it:

>>rm ~/.foundry/keystores/defaultKey

Key Related Other Commands

Verify the Existing Key:

```
cast wallet list
```

Overwrite the Existing Key:

```
rm /home/ksh/.foundry/keystores/defaultKey
```

Use a Different Key Name:

```
cast wallet import newKeyName --interactive
```

--interactive flag

```
ksh@DESKTOP-448NKKT:/mnt/c/Users/ksriv$ cast wallet import defaultKey --interactive
Enter private key:
Enter password:
`defaultKey` keystore was saved successfully. Address: 0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266
```

Example Without `--interactive`:

Without the `--interactive` flag, the command executes immediately, without any prompts. This is useful for automation or when you're confident about the deployment details.

```bash
forge create Counter --rpc-url http://127.0.0.1:8545
```

Summary

The `--interactive` flag is used to add **human oversight** to the process, making it more controlled and ensuring that the user confirms every significant action before proceeding. This is essential when deploying contracts, especially in a live or test environment, where one mistake can be costly.

1. Confirming Actions

The flag prompts you for confirmation before continuing, ensuring that you have an opportunity to review and approve every step of the deployment process.

- For example, it will ask you to confirm the contract you're deploying, the sender's address, gas fees, and other important details.

```bash
forge create Counter --rpc-url http://127.0.0.1:8545 --interactive
```

With the `--interactive` flag, Foundry may ask questions like:

- "Do you want to deploy this contract?"
- "Do you want to use this address as the sender?"
- "Confirm the gas price and other parameters."

2. **Selecting Wallet/Account**

If you have multiple wallets or keys, Foundry will allow you to **select** the one you'd like to use interactively. It’s helpful when you don’t want to hardcode private keys or wallet addresses in the command.

3. **Step-by-Step Process**

The flag makes the deployment process more **manual** and slow, as it will wait for you to provide inputs or confirmations at each step. This can help prevent accidental transactions or wrong configurations.

4. **Deployment Testing/Simulation**

If you're deploying in a test environment or want to ensure everything is correct before proceeding, the interactive flag gives you a chance to verify everything before confirming the actual transaction.

Installing Packages using forge

To install a package

```
forge install package-name
```

To get the packages installed

```
forge install remappings
```

To update a package

```
forge update package-path
```

To remove a package

```
forge remove package-name
```

Installing solmate

```
forge install transmissions11/solmate
```

- lib
  - forge-std
  - solmate

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.13;

import "solmate/tokens/ERC20.sol";

contract CTE is ERC20("CodeEater","symbol",18){

}
```

>>>forge install transmissions11/solmate

Installing Packages using npm

To install a package using npm

```
npm install package-name
```

To install a package using npm

```
npm install @openzeppelin/contracts
```

node_modules
- @openzeppelin
- .package-lock.json

```
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
contract CTE is ERC20("CodeEater","symbol"){}
```

remappings.txt

```
1    @openzeppelin/=node_modules/@openzeppelin/
```

>>>npm install @openzeppelin/contracts (here manual import the path)

Installing Packages using npm

To install a package using npm

```
npm install package-name
```

remappings.txt

```
@openzeppelin/ = node_modules/@openzeppelin
```

Storage layout of a contract

```
forge inspect
node_modules/@openzeppelin/contracts/governance/Governor.sol:Governor storage
--pretty
```

```
| Name            | Type                                          | Slot | Offset | Bytes |
                                |
                                |
|-----------------|-----------------------------------------------|------|--------|-------|
|-----------------|-----------------------------------------------|------|--------|-------|
                                |
                                |
|-----------------|-----------------------------------------------|------|--------|-------|
----------------------------|
| _nameFallback   | string                                        | 0    | 0      | 32    |
----------------------------|
----------------------------|
| _nameFallback   | string                                        | 0    | 0      | 32    |
| _versionFallback | string                                       | 1    | 0      | 32    |
| _nonces         | mapping(address => uint256)                   | 2    | 0      | 32    |
| _nameFallback   | string                                        | 0    | 0      | 32    |
| _versionFallback | string                                       | 1    | 0      | 32    |
| _nonces         | mapping(address => uint256)                   | 2    | 0      | 32    |
n/contracts/governance/Governor.sol:Governor |
| _nonces         | mapping(address => uint256)                   | 2    | 0      | 32    |
| _name           | string                                        | 3    | 0      | 32    |
 node_modules/@openzeppelin/contracts/governance/Governor.sol:Governor |
| _name           | string                                        | 3    | 0      | 32    |
| _proposals      | mapping(uint256 => struct Governor.ProposalCore) | 4    | 0      | 32    |
n/contracts/governance/Governor.sol:Governor |
 node_modules/@openzeppelin/contracts/governance/Governor.sol:Governor |
| _governanceCall | struct DoubleEndedQueue.Bytes32Deque          | 5    | 0      | 64    |
 node_modules/@openzeppelin/contracts/governance/Governor.sol:Governor |
```

>>>forge inspect node_modules/@openzeppelin/contracts/governance/Governor.sol:Governor storage

Methods of a contract

```
forge inspect
node_modules/@openzeppelin/contracts/governance/Governor.sol:Governor
methods --pretty
```


```
{
  "BALLOT_TYPEHASH()": "deaaa7cc",
  "CLOCK_MODE()": "4bf5d7e9",
{
  "BALLOT_TYPEHASH()": "deaaa7cc",
  "CLOCK_MODE()": "4bf5d7e9",
  "COUNTING_MODE()": "dd4e2ba5",
  "COUNTING_MODE()": "dd4e2ba5",
  "EXTENDED_BALLOT_TYPEHASH()": "2fe3e261",
  "cancel(address[],uint256[],bytes[],bytes32)": "452115d6",
  "castVote(uint256,uint8)": "56781388",
  "castVoteBySig(uint256,uint8,address,bytes)": "8ff262e3",
  "castVoteWithReason(uint256,uint8,string)": "7b3c71d3",
  "castVoteWithReasonAndParams(uint256,uint8,string,bytes)": "5f398a14",
  "castVoteWithReasonAndParamsBySig(uint256,uint8,address,string,bytes,bytes)": "5b8d0e0d",
  "clock()": "91ddadf4",
  "eip712Domain()": "84b0196e",
  "execute(address[],uint256[],bytes[],bytes32)": "2656227d",
  "EXTENDED_BALLOT_TYPEHASH()": "2fe3e261",
  "cancel(address[],uint256[],bytes[],bytes32)": "452115d6",
  "castVote(uint256,uint8)": "56781388",
  "castVoteBySig(uint256,uint8,address,bytes)": "8ff262e3",
  "castVoteWithReason(uint256,uint8,string)": "7b3c71d3",
  "castVoteWithReasonAndParams(uint256,uint8,string,bytes)": "5f398a14",
  "castVoteWithReasonAndParamsBySig(uint256,uint8,address,string,bytes,bytes)": "5b8d0e0d",
  "clock()": "91ddadf4",
  "eip712Domain()": "84b0196e",
  "execute(address[],uint256[],bytes[],bytes32)": "2656227d",
```


>>>forge inspect node_modules/@openzeppelin/contracts/governance/Governor.sol:Governor methods

Methods of a contract

```
forge inspect
node_modules/@openzeppelin/contracts/governance/Governor.sol:Governor abi
--pretty
```

```
library IGovernor {
    type ProposalState is uint8;
}

interface Governor {
    error FailedCall();
    error GovernorAlreadyCastVote(address voter);
    error GovernorAlreadyQueuedProposal(uint256 proposalId);
    error GovernorDisabledDeposit();
    error GovernorInsufficientProposerVotes(address proposer, uint256 votes, uint256 threshold);
    error GovernorInvalidProposalLength(uint256 targets, uint256 calldatas, uint256 values);
    error GovernorInvalidSignature(address voter);
    error GovernorInvalidVoteParams();
    error GovernorInvalidVoteType();
    error GovernorInvalidVotingPeriod(uint256 votingPeriod);
    error GovernorNonexistentProposal(uint256 proposalId);
    error GovernorNotQueuedProposal(uint256 proposalId);
    error GovernorOnlyExecutor(address account);
    error GovernorOnlyProposer(address account);
    error GovernorQueueNotImplemented();
    error GovernorRestrictedProposer(address proposer);
    error GovernorUnexpectedProposalState(uint256 proposalId, IGovernor.ProposalState current, bytes32 expectedStates);
    error InvalidAccountNonce(address account, uint256 currentNonce);
    error InvalidShortString();
    error SafeCastOverflowedUintDowncast(uint8 bits, uint256 value);
    error StringTooLong(string str);

    event EIP712DomainChanged();
    event ProposalCanceled(uint256 proposalId);
    event ProposalCreated(
```

>>>forge inspect node_modules/@openzeppelin/contracts/governance/Governor.sol:Governor abi

Testing With Foundry

Module - 2

Hello! Foundry

```
contract HelloFoundry is Test {

    function setUp() public {
    }

    function testMeFoundry() public pure {
        console.log("testMeFoundry");
    }
    function meFoundry() public pure {
        console.log("meFoundry");
    }
}
```

This will be executed because foundry executes functions which starts with "test"

This will not be executed.

Hello World Test

```
forge test --match-path test/HelloWorld.t.sol -vvv
```

```
contract HelloWorld {
    string public str="Hello World";
}
```

the `setUp()` function in Foundry is used for initializing state that will be reused across multiple test functions. This might include things like setting up contract instances, initializing variables, or setting balances.

```
contract HelloWorldTest is Test {
    HelloWorld public helloWorld;

    function setUp() public {
        helloWorld = new HelloWorld();
    }

    function test_str() public view {
        assertEq(helloWorld.str(),"Hello World");
    }

}
```

>>>forge test --match-path test/HelloWorld.t.sol
—set up function har test ke pehle run hoga(har naye instance pe call hoga)
>>>-vvv (check console)

Verbosity

```
ksh@DESKTOP-448NKKT:/mnt/c/Projects/foundry/foundry-demo$ forge test --match
-path test/HelloWorld.t.sol -vv
[⠊] Compiling...
No files changed, compilation skipped

Ran 1 test for test/HelloWorld.t.sol:HelloWorldTest
[FAIL: assertion failed: Hello World != Fello World] test_str() (gas: 12333)
Suite result: FAILED. 0 passed; 1 failed; 0 skipped; finished in 241.17µs (6
2.34µs CPU time)

Ran 1 test suite in 3.18ms (241.17µs CPU time): 0 tests passed, 1 failed, 0
skipped (1 total tests)

Failing tests:
Encountered 1 failing test in test/HelloWorld.t.sol:HelloWorldTest
[FAIL: assertion failed: Hello World != Fello World] test_str() (gas: 12333)

Encountered a total of 1 failing tests, 0 tests succeeded
```

Reason of why the test failed(-vv)

```
ksh@DESKTOP-448NKKT:/mnt/c/Projects/foundry/foundry-demo$ forge test --match
-path test/HelloWorld.t.sol -vvv
[⠊] Compiling...
[⠒] Compiling 1 files with Solc 0.8.28
[⠃] Solc 0.8.28 finished in 353.37ms
Compiler run successful!

Ran 1 test for test/HelloWorld.t.sol:HelloWorldTest
[FAIL: assertion failed: Hello World != Fello World] test_str() (gas: 12333)
Traces:
  [12333] HelloWorldTest::test_str()
    ├─ [3142] HelloWorld::str() [staticcall]
    │   └─ ← [Return] "Hello World"
    ├─ [0] VM::assertEq("Hello World", "Fello World") [staticcall]
    │   └─ ← [Revert] assertion failed: Hello World != Fello World
    └─ ← [Revert] assertion failed: Hello World != Fello World
 0 tests passed, 1 failed, 0 skipped (1 total tests)

Failing tests:
Encountered 1 failing test in test/HelloWorld.t.sol:HelloWorldTest
[FAIL: assertion failed: Hello World != Fello World] test_str() (gas: 12333)

Encountered a total of 1 failing tests, 0 tests succeeded
```

Reason of why the test failed(-vvv)

Test fail

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.10;

contract Counter {
    uint256 public count;

    function increment() public {
        count += 1;
    }

    function decrement() public {
        require(count > 0, "Counter cannot be negative");
        count -= 1;
    }

    function getCount() public view returns (uint256) {
        return count;
    }
}
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.10;
import {Test} from "forge-std/Test.sol";
import "../src/Counter.sol"; // Import the contract to test

contract CounterTest is Test {
    Counter counter;

    function setUp() public {
        counter = new Counter(); // Deploy new contract before each test
    }

    function testRevert_DecrementBelowZero() public {
        vm.expectRevert("Counter cannot be negative"); // Expect failure
        counter.decrement(); // This should fail because count is 0
    }

    function testRevert_SubtractTwice() public {
```

```
        counter.increment(); // Count = 1
        counter.decrement(); // Count = 0
        counter.increment(); // Count = 1
        counter.increment(); // Count = 2
        vm.expectRevert("Counter cannot be negative"); // Expect failure
        counter.decrement(); // Should fail because count is 0 again
    }
}
```

//whenever if i know that it will fail i use vm.expectRevert
//and if fail then test pass and if fail not happen then test fail
//also if you pass the message in vm.expectRevert(“message”) and message not match with base code file and test file then test is failed.

Example Workflow (Step by Step):

1. `forge test` starts.
2. Foundry **deploys HelloWorld.sol** to `0xabc...123` (example address).
3. Foundry **deploys HelloWorldTest.sol** to `0xdef...456`.
4. In `setUp()`, `instance = new HelloWorld();` is called:
   - Deploys HelloWorld again to a unique address (like `0xabc...789`).
5. Now, when `instance.getStr()` is called:
   - The call comes from `0xdef...456` (the test contract).
   - `msg.sender` inside HelloWorld is `0xdef...456`.
6. Foundry captures all **gas usage**, **call details**, and **test results**.

In Short:

- **HelloWorld.sol** is your main contract.
- **HelloWorldTest.sol** is your test contract.
- Foundry deploys both and makes them interact.
- `msg.sender` is the address calling the function (the test contract by default).
- You can simulate different senders using `vm.prank`.
- All **calls** and **gas usage** are tracked by Foundry during the process.

Jo test contract hai woh bhi solidity hi Hai to jab test contract bhi solidity Hai to yeh bhi Dhyan rakhna hoga ki solidity contract ke function ko bhi koi call Kara raha Hai.to in solidity contract ke function ko koi call Kara raha Hai to Sidhi si bat Hai ki jo hame value milne vali Hai message.sender ki woh test contract ki hai.

🔍 Does `vm.prank()` or `vm.startPrank()` Create a New Instance?

❌ **No, `vm.prank()` and `vm.startPrank()` do not create a new contract instance.**

- They **only change** `msg.sender` for the next transaction(s).
- The contract instance remains the same during the test execution.

✅ **What Actually Happens?**

- `vm.startPrank(address(1));` makes **all following calls** come from `address(1)`.
- `vm.stopPrank();` resets `msg.sender` back to the default test contract address.

Prank & startPrank

The **prank** cheat code is used to mock a single call or transaction as being sent from a specified address. It is set up just before making a function call, and it only affects that one transaction.

The **startPrank** cheat code is used to mock multiple calls or transactions as being sent from a specified address. This is useful if you want a series of calls or transactions to be performed under the same sender context. The effect persists until you call vm.stopPrank().

Example(Prank) Owner.sol - Click Here

Example(startPrank) Multicaller.sol - Click Here

Gas report

```
forge test --match-path test/HelloWorld.t.sol --gas-report
```

Deployment Cost	Deployment Size				
126969	373				
Function Name	**min**	**avg**	**median**	**max**	**# calls**
divByZero	2339	2339	2339	2339	1
increment	43438	43438	43438	43438	2
increment	43438	43438	43438	43438	2
number	283	283	283	283	258
setNumber	23582	43081	43566	43866	260
setNumber	23582	43081	43566	43866	260

Understanding the gas consumption of test functions in Solidity development can indeed seem unnecessary at first glance. However, By analyzing gas usage during tests, you can identify specific functions or operations that are more gas-intensive than expected.

The testing framework or configuration is set to rerun tests multiple times gas profiling.

>>> forge test --match-path test/HelloWorld.t.sol --gas-report
>>>forge test --gas-report --optimize

Statistics

Min	The minimum amount of gas used in any single execution of this function.
Avg	The average gas consumption across all executions.
Median	The middle value of gas consumption if all executions were sorted by cost.
Max	The maximum gas used in any execution.

What is `calls`?

`calls` means **how many times** a function is used during testing.

Real-Life Example:

Imagine you're testing how much fuel a car uses for short trips:

- You drive the car **5 times** for 10 km each trip.
- After each trip, you measure the fuel used.

Here:

- **Each trip = 1 call.**
- After 5 trips, you can find the **average, minimum, maximum fuel usage**, etc.

In Smart Contracts:

If your contract function is called **5 times during tests**, the gas report shows:

- **Min gas used in 1 call**
- **Max gas used in 1 call**
- **Average gas used across all 5 calls**
- **#Calls** = 5 (how many times it was tested)

So, **more calls = better accuracy** for gas usage!

Does that help? 😊

Gas Statistics

Call 1	Call 2	Call 3	Call 4	Call 5
40,000 gas	42,000 gas	41,000 gas	40,500 gas	43,000 gas

Min - 40,000 gas

Max - 43,000 gas

Avg - 41,300 gas

Med - 41,000 gas

(40 000 + 43 000 + 42 000 + 41 000 + 40 500) / 5 =
41300

Contract - Console.log

```
forge test --match-path test/Counter.t.sol -vv
```

```
import "forge-std/console.sol";

contract Counter {
    uint256 public number;

    function setNumber(uint256 newNumber) public {
        number = newNumber;
    }

    function increment() public {
        console.log("Number Before Increment",number);
        number++;
        console.log("Number After Increment",number);
    }
```

Test File - Console.log

```
function testLogCounter() public view{
    console.log("Counter Value");
    uint value = counter.number();
    console.logUint(value);
}
```

Error

RequireRevert - Click Here

Deal

In Foundry, `deal` is a utility function used to directly set or manipulate the balance of an account for any token or native cryptocurrency within a test. This is especially useful for testing scenarios that require specific token balances, like checking edge cases or functionality that depends on having a minimum balance, without needing to execute transfers or use a faucet.

For example, in tests on a mainnet fork, `deal` allows you to set a wallet's balance for an ERC-20 token (e.g., DAI or USDC) to a desired amount. This makes it much faster to set up test conditions and is particularly valuable when interacting with tokens already deployed on the mainnet.

deal(address token, address user, uint256 amount, bool transfer)

`address token`: The token address (e.g., USDC, DAI).

`address user`: The address whose balance you want to modify.

`uint256 amount`: The amount you want to set for the user.

`bool transfer`: Whether to simulate the balance change as if it were a transfer.

Deal.t.sol - Click Here

Fork

```
forge test --fork-url
https://eth-mainnet.g.alchemy.com/v2/xhB6Ea_CdRJ9mt7_gExxwayO0v5qAPdP
--match-path test/Deal.t.sol -vvv
```

DAI.t.sol - Click Here

>>>forge test --fork-url https://eth-mainnet.g.alchemy.com/v2/pe6oK3SEKpr--ifwco_vcjwLv0oRqCWW --match-path test/Deal.t.sol -v

FFI

FFI (Foreign Function Interface) in Foundry allows smart contracts to interact with external systems during tests by executing shell commands. It enables running scripts or programs written in any language, such as Python or Bash, and using their outputs within Solidity tests.

To use FFI, include **vm.ffi()** in your tests and enable FFI with forge test **--ffi**. It's powerful for testing scenarios like interacting with off-chain systems but should be used cautiously due to security risks and reproducibility concerns.

FFI Assignment

Execute the following script: **bash -c date +%s**
This script fetch the system timestamp.

>>>forge test --ffi --match-path test/FFI.t.sol -vvv

Event Testing

These flags indicate whether to check the indexed and non-indexed parameters of the event

```
vm.expectEmi(bool checkTopic1,bool checkTopic2,bool checkTopic3, bool checkData);
```

Example - Event.sol

here forth is non check data means here checking any non indexed value am i right?

Yes, you are absolutely right! ✅

The **fourth boolean** (`true`) in

```solidity
vm.expectEmit(true, false, false, true);
```

means checking any non-indexed value in the event.

Breaking it Down:

In this event:

```solidity
event ValueChanged(address indexed sender, uint256 indexed oldValue, uint256 newValue);
```

- `sender` → **Indexed** ✅
- `oldValue` → **Indexed** ✅
- `newValue` → **Not Indexed** ❌ (Stored in log data, not topics)

How `vm.expectEmit(true, false, false, true)` Works:

Boolean	Meaning
`true` (1st)	Check the first **indexed** parameter (`sender`)
`false` (2nd)	Ignore the second **indexed** parameter (`oldValue`)
`false` (3rd)	Ignore the third **indexed** parameter (`newValue` , if indexed)
`true` (4th)	**Check the non-indexed** `newValue`

Conclusion

✅ **Fourth** `true` **checks non-indexed values.**

✅ In this case, it ensures that `newValue` (42 in the test) **matches the expected value.**

You're getting it perfectly! 🚀 Keep going! 🎯

here only indexedvalur is checking if i do true then

Not exactly! If you **set** `true` **for an indexed value**, it means you **are checking that specific indexed parameter** in the event logs.

Let's break it down again.

Your Code Example:

```solidity
vm.expectEmit(true, false, false, true);
```

- **First** `true` → ✅ Check the **first indexed parameter** (`sender`)
- **Second** `false` → ❌ Ignore the **second indexed parameter** (`oldValue`)
- **Third** `false` → ❌ Ignore the **third indexed parameter** (`newValue` , if indexed)
- **Fourth** `true` → ✅ **Check the non-indexed data** (`newValue` , **because it's stored in log data)**

If You Change It to:

```solidity
vm.expectEmit(true, true, true, false);
```

- ✅ Check `sender` (indexed)
- ✅ Check `oldValue` (indexed)
- ✅ Check `newValue` (if indexed)
- ❌ **Ignore non-indexed values**

So, **if you set** `true` **for an indexed value**, the test will verify that the emitted event has the exact expected value for that indexed field.

But if you set `false` **, it ignores that indexed field.** 🚀

Let me know if you need more clarity! 💡

Advance Testing With Foundry

Module - 3

Fuzz Testing

Fuzzing (or fuzz testing) is a technique where random, unexpected, or malformed data is automatically generated and input into a program to see if it causes crashes, unexpected behaviors, or vulnerabilities.

Purpose: Fuzz testing is designed to find potential issues by testing the contract with a wide range of random inputs. The main goal is to check if the contract behaves as expected across various unexpected inputs.

Methodology:

- Fuzz testing involves generating random inputs for functions to simulate unpredictable user interactions.
- Foundry automatically generates inputs for functions based on specified parameters, helping developers uncover edge cases and potential bugs that wouldn't normally be tested.
- This can reveal unexpected behavior or errors such as reverts, assertions, or state changes that violate the expected outcomes.

Fuzz Testing Example

Example: If a function is supposed to handle token transfers, fuzz testing would supply various random values to test how the function handles extreme values, such as zero, maximum token amount, or values beyond typical usage.

```
function testFunction(uint amount) public {
   // This is a fuzz test where Foundry will
   //automatically test the function
   // with various random `amount` values
}
```

Fuzz Testing

```
Ran 2 tests for test/Safe.t.sol:SafeTest
[PASS] testFuzz_Withdraw(uint96) (runs: 257, μ: 22873, ~: 22873)
[PASS] test_Withdraw() (gas: 22286)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 11.94ms (9.65ms CPU time)
```

1) "runs" refers to the amount of scenarios the fuzzer tested.
   By default, the fuzzer will generate 256 scenarios, but this and other test execution parameters can be setup by the user.
2) Fuzzer configuration details are provided here. "μ" (Greek letter mu) is the mean gas used across all fuzz runs.
3) "~" (tilde) is the median gas used across all fuzz runs

Invariant Testing

Invariant testing in Foundry is a test methodology that uses stateful fuzzing to verify the correctness of code.An invariant test is a stateful fuzz test, which means that the state of the test at one call is saved for the next call.

Purpose: Invariant testing is used to verify that certain properties (or invariants) always hold true throughout the contract's lifecycle, regardless of the inputs or the sequence of function calls.

Methodology:

- In invariant testing, you define conditions that should always be true, such as "total supply of tokens should never decrease" or "the sum of all balances should always equal the total supply."
- Foundry repeatedly runs various interactions with the contract (including state-changing functions) to check if these conditions hold under all possible states.
- Invariant tests ensure the contract's integrity over long-term use and help verify that core properties remain stable regardless of how the contract is used.

Invariant Testing Example

Invariant testing fails at function sev() since votingStatus is assigned true.

```
contract InvariantExample {
    bool public votingStatus;

    function one() external{}
    function two() external{}
    function three() external{}
    function four() external{}
    function five() external{}
    function six() external{}
    function sev() external{
        votingStatus=true;
    }
    function eig() external{}
    function nin() external{}
    function ten() external{}
}
```

```
import {InvariantExample} from "../src/InvariantExample.sol";

contract InvariantExampleTest is Test {
    InvariantExample private target;

    function setUp() public {
        target = new InvariantExample();
    }

    function invariant_votingStatus_never_true() public view {
        assertEq(target.votingStatus(), false);
    }
}
```

```
[FAIL: invariant_votingStatus_never_true replay failure]
        [Sequence]
                sender=0x000000000000000000000000000000000000008b addr=[src/InvariantExample.sol:InvariantExample]0x5615dEB798BB3E4dFa0139dFa1b3D433Cc23b72f calldata=sev() args=[]
```

Differential Testing

Differential Testing in Foundry involves comparing the results of multiple implementations of the same functionality to ensure they behave identically under all scenarios.

It is particularly useful when testing for :

- Consistency across versions of the same contract.
- Consistency between different implementations of a function or algorithm.
- Identifying regressions when refactoring code.

Cast

Cast is Foundry's command-line tool for performing Ethereum RPC calls. You can make smart contract calls, send transactions, or retrieve any type of chain data - all from your command-line!

```
cast call
0x6b175474e89094c44da98b954eedeac495271d0
f "totalSupply()(uint256)" --rpc-url
https://eth-mainnet.alchemyapi.io/v2/Lc7o
IGYeL_QvInzI0Wiu_pOZZDEKBrdf
```

cast Commands

- General Commands
- Chain Commands
- Transaction Commands
- Block Commands
- Account Commands
- ENS Commands
- Etherscan Commands
- ABI Commands
- Conversion Commands
- Utility Commands
- Wallet Commands

✅ `forge compile` — What It Does

The `forge compile` command compiles all your Solidity smart contracts in a Foundry project using the configured Solidity compiler (`solc`) version.

🔍 What Happens When You Run `forge compile`

1. **Reads your project config** from:
   - `foundry.toml` (compiler version, remappings, etc.)
2. **Finds all `.sol` files** under:
   - `src/` (default source folder)
3. **Compiles** all contracts:
   - Resolves imports using remappings (e.g., `forge-std` , `solmate`)
   - Optimizes if enabled
4. **Outputs artifacts** into:
   - `out/` folder (compiled contract `.json` files)
   - `cache/` folder (internal build metadata for fast recompilation)

📦 Output Structure

After running `forge compile`, you'll get:

`out/`

Contains build artifacts:

```bash
out/
  Counter.sol/
    Counter.json          # ABI, bytecode, deployed bytecode, etc.
```

`cache/`

Used internally by Foundry for incremental builds:

```bash
cache/
  build-info/           # solc intermediate data
  cache.json            # internal compiler cache info
```

Great observation — and the reason is important to understand:

❓ Why does `forge create` not generate a `broadcast/` folder?

Because:

> `forge create` **is a low-level deployment command, not a script**, and it **does not go through the scripting engine**, which is what generates the `broadcast/` folder.

🔍 `forge script` vs `forge create`

Feature	`forge script`	`forge create`
Deploy contract?	✅ Yes	✅ Yes
Simulate (dry run)?	✅ Yes (without `--broadcast`)	❌ No simulation — real tx only
Write to `broadcast/`?	✅ Yes (`run-latest.json`, `broadcast-latest.json`)	❌ No broadcast folder
Can call methods?	✅ Yes (you can call methods in `run()`)	❌ No, just deploy
Recommended for?	Full deployment & scripting workflows	Quick, one-off contract deployment

agar broadcast karega to nonce increase hoga like 1 and then 2 then 3 like that.agar tu broadcast remove karega to dry run me nonce 4 ayega.and tu jitni bhi bar without broadcast karega nonce 4 hi rahega.agar tu firs se broadcast use karega to waha pe to 4 se hi nonce continue hoga.strange behaviour in dry run and expected behaviour in broadcast

12:08

🧠 Summary

- `checkData = true` tells Foundry to **match all non-indexed args**.
- In your case, it checks **both** `oldValue` **and** `newValue` **together**.
- You **cannot** selectively check just one of them using this — it always checks **all non-indexed fields** if `checkData == true`.

If you want to match **only one**, you'd need to restructure the event.

Let me know if you want an example where only one parameter is checked!

suppose i want to check from both only then i have to make any of them is indexed am i right?

Exactly — you're 100% right ✅

If you want to selectively check **only one of the parameters**, then you need to:

suppose i want to check from both only then i have to make any of them is indexed am i right?

Exactly — you're 100% right ✅

If you want to selectively check **only one of the parameters**, then you need to:

✅ Make that parameter `indexed` in the event.

Why?
Because in Solidity:

- **Indexed** parameters go into `topics` (can be individually checked in `vm.expectEmit`)
- **Non-indexed** parameters go into `data` (checked **all at once** if `checkData == true`)

🔍 Example:

Original event:

```solidity
event ValueChanged(address indexed user, uint256 oldValue, uint256 newValue);
```

All data fields (`oldValue`, `newValue`) are **non-indexed** — you can't check them individually.

✅ Modified version:

If you want to check **only** `newValue`, index it:

```solidity
event ValueChanged(address indexed user, uint256 oldValue, uint256 indexed newValue);
```

Now:

- `user` → topic1
- `newValue` → topic2
- `oldValue` → data

So in the test:

```solidity
vm.expectEmit(true, false, true, false);
emit ValueChanged(address(this), 0, 42);
```

This tells Foundry:

- ✅ Check `user` (topic1)
- ❌ Skip `oldValue` (in data, since `checkData = false`)
- ✅ Check `newValue` (topic2, because we marked it `indexed`)

→ Now only `user` and `newValue` will be compared.

🔐 Limitations

- You can have **up to 3** `indexed` **parameters** in any event
- Only `indexed` parameters can be checked **individually**
- `non-indexed` parameters must be **all or none** (`checkData = true` or `false`)

🧠 Summary Table

Field	Indexed?	Stored In	Can check individually?
`user`	✅	topic1	✅ yes (via `checkTopic1`)
`newValue`	✅	topic2	✅ yes (via `checkTopic2`)
`oldValue`	❌	data	❌ no — only via `checkData == true`